

Implementation

5.1 Database Setup

Create / Use Database

```
mysql> USE SalesDB;
```

```
Database changed
```

5.2 Table Creation

Create Sales Table

```
mysql> CREATE TABLE Sales (  
->     SaleID      INT AUTO_INCREMENT PRIMARY KEY,  
->     ProductName VARCHAR(50),  
->     Quantity    INT,  
->     Price       INT  
-> );
```

```
Query OK, 0 rows affected (0.01 sec)
```

5.3 Stored Procedure Implementation

Procedure to Insert Sales Data — Syntax of Procedure (AddSale)

```
mysql> DELIMITER $$
```

```
mysql> CREATE PROCEDURE AddSale(  
->     IN p_name  VARCHAR(50),  
->     IN p_qty   INT,  
->     IN p_price INT  
-> )  
-> BEGIN  
->     INSERT INTO Sales(ProductName, Quantity, Price)  
->     VALUES (p_name, p_qty, p_price);  
-> END$$
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
```

5.4 Procedure Execution

Calling the Stored Procedure — Inserting Sample Records

```
mysql> CALL AddSale('Laptop', 2, 50000);
```

```
Query OK, 1 row affected (0.00 sec)
```

```
mysql> CALL AddSale('Mobile', 5, 20000);
```

```
Query OK, 1 row affected (0.01 sec)
```

```
mysql> CALL AddSale('Tablet', 3, 15000);
```

```
Query OK, 1 row affected (0.00 sec)
```

5.5 Data Display

Display Sales Table Data

```
mysql> SELECT * FROM Sales;
```

```
+-----+-----+-----+-----+
| SaleID | ProductName | Quantity | Price |
+-----+-----+-----+-----+
|      1 | Laptop      |         2 | 50000 |
|      2 | Mobile      |         5 | 20000 |
|      3 | Tablet      |         3 | 15000 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

5.6 Function Implementation

Function to Calculate Total Sales — Logic using SUM()

```
mysql> DELIMITER $$
```

```
mysql> CREATE FUNCTION TotalSales()
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->     DECLARE total INT;
->
->     SELECT SUM(Quantity * Price) INTO total FROM Sales;
->
->     RETURN total;
-> END$$
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> DELIMITER ;
```

5.7 Function Execution

Calling Function to Get Total Sales

```
mysql> SELECT TotalSales() AS Total_Sales;
```

```
+-----+
| Total_Sales |
+-----+
```

```
|          245000 |
+-----+
1 row in set (0.00 sec)
```

5.8 Advanced Function (Optional)

Function for Customer/Product-wise Sales — Conditional Filtering Logic

```
mysql> DELIMITER $$
```

```
mysql> CREATE FUNCTION get_total_sales()
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->     DECLARE vttotal INT;
->
->     SELECT SUM(amount) INTO vttotal FROM sales;
->
->     IF vttotal IS NULL THEN
->         SET vttotal = 0;
->     END IF;
->
->     RETURN vttotal;
-> END$$
```

```
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> DELIMITER ;
```

```
mysql> SELECT get_total_sales();
```

```
+-----+
| get_total_sales() |
+-----+
|          750000 |
+-----+
1 row in set (0.00 sec)
```

Function for Customer/Product-wise Sales

```
mysql> DELIMITER $$
```

```
mysql> CREATE FUNCTION get_total_sale1(
->     p_cname      VARCHAR(30),
->     p_productname VARCHAR(30)
-> )
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->     DECLARE vttotal INT;
->
->     SELECT SUM(amount)
->     INTO vttotal
->     FROM sales
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> SELECT get_total_sale1('Rama', NULL);
```

```
1 row in set (0.00 sec)
```